helixqa-jackett-fake

helixqa_jackett_fake_behavioral_equivalence_challenge.sh

Revision: 1 **Last modified:** 2026-08-15T12:00:00Z **Status:** active **Maintainer:** Track 11 — BOB-067 Lava-P4 port **Scope:** anti-bluff regression guard for the behaviorally-equivalent Jackett fake used by the boba autonomous test regime

Overview

The Lava-P4 porting directive is explicit: *“the test fake MUST 302-without- cookie like real Jackett (behavioral equivalence) or the gap stays hidden.”* This challenge is the mechanized self-validation guard proving the fake never degrades into a bluff-fake (a fake that would 200 on apikey-only management and hide the exact defect the fake was built to catch).

Fake location

The fake is embedded in `qBitTorrent-go/internal/jackett/cookie_login_test.go` as `newPasswordProtectedJackett()` (Go `httptest.Server`). Staying in `_test.go` keeps it out of every production import graph (§11.4.27 no-fakes- beyond-unit-tests) while remaining consumable by every test in the same package.

Behavioral contract (what the fake MUST reproduce)

Cross-checked LIVE against real Jackett at `http://localhost:9117` on 2026-08-15 (values captured verbatim from `curl -s -D-`):

Behavior	Real Jackett (live)	Fake
`GET /api/v2.0/indexers?`		`HTTP 302` + `Location: /UI/Login`

Behavior	Real Jackett (live)	Fake
`apikey=<bogus>` (no cookie)	`HTTP/1.1 302` + `Location: .../UI/Login?ReturnUrl=%2Fapi%2Fv2.0%2Findexers...`	
`POST /UI/Dashboard password=` (correct/empty)	`HTTP/1.1 302` + `Set-Cookie: Jackett=CfDJ8...` + `Location: Dashboard`	`HTTP 302` + `Set-Cookie: Jackett=valid-session-token` + `Location: /`
`POST /UI/Dashboard password=<wrong>`	`HTTP/1.1 200` re-renders login HTML, NO `Set-Cookie`	`HTTP 200` + `<html>login</html>`, NO `Set-Cookie`
`GET /api/v2.0/indexers` WITH valid session cookie	`HTTP/1.1 200` JSON catalog	`HTTP 200` + fixture JSON `[{"id":"rutracker",...}]`

The cookie name `Jackett` matches real Jackett exactly (verified live).

Prerequisites

- `go` toolchain on `PATH` (challenge SKIPs with reason if absent).
- `qBitTorrent-go/internal/jackett/cookie_login_test.go` present.
- OPTIONAL: a live Jackett reachable at `$JACKETT_LIVE_URL` (default `http://localhost:9117`). If reachable, the challenge diffs the fake's contract against the live product on B1 (apikey-only→302) and B2 (login→302+`Set-Cookie: Jackett=`). If not reachable, that cross-check is honestly SKIPped with an explicit `[UNCONFIRMED-AGAINST-LIVE-JACKETT]` marker in the output.

Usage

```
# GREEN — shipped regression guard
RED_MODE=0 bash challenges/scripts/
        helixqa_jackett_fake_behavioral_equivalence_challenge.sh

# GREEN with an explicit live target
JACKETT_LIVE_URL=http://jackett.internal:9117 \
    bash challenges/scripts/
        helixqa_jackett_fake_behavioral_equivalence_challenge.sh

# RED — reproduces the bluff-fake / absent-fake state (PASS if
        degraded)
RED_MODE=1 bash challenges/scripts/
        helixqa_jackett_fake_behavioral_equivalence_challenge.sh
```

What it proves

1. Runs `TestFakeJackettRefusesManagementWithoutCookie` (already embedded next to the fake) — the load-bearing anti-bluff guard that a bare `GET /api/v2.0/indexers?apikey=...` (no cookie) returns 302, not 200.
2. Asserts the pass line is present with `-v` output (§11.4.201 false-null guard: a mis-typed `-run` filter also returns exit 0 with no test).
3. When a live Jackett is reachable, cross-checks the fake's B1 + B2 against the real product using raw `curl` (independent transport, no shared code path with the fake or the client-under-test).

Falsifiability rehearsal (paired §1.1 mutation)

The sibling `jackett_cookie_login_hardening_challenge.sh` doc describes a `doManaged`-side mutation that made the challenge FAIL. Under this challenge, the analogous fake-side mutation is: remove the `hasValidSession(r)` gate from the management branch so the fake returns 200 on apikey-only management. `TestFakeJackettRefusesManagementWithoutCookie` catches that mutation directly and this challenge exits 1.

Live evidence (2026-08-15, localhost:9117)

```
$ curl -s -D- -o /dev/null --max-time 3 \
    'http://localhost:9117/api/v2.0/indexers?apikey=bogus-
key&configured=false'
HTTP/1.1 302 Found
Location: http://localhost:9117/UI/Login?
ReturnUrl=%2Fapi%2Fv2.0%2Findexers%3Fapikey%3Dbogus-
key%26configured%3Dfalse

$ curl -s -D- -o /dev/null --max-time 3 -X POST -d 'password=' \
    http://localhost:9117/UI/Dashboard
HTTP/1.1 302 Found
Location: Dashboard
Set-Cookie: Jackett=CfDJ8J4t0dXrfJhAr2dBxs4c...; path=/;
samesite=lax; httponly
```

The fake's `Location`/`Set-Cookie` shape matches — behavioral equivalence is PROVEN LIVE, not asserted.

Honest boundary (§11.4.6)

- Live-check verifies B1 + B2. B3 (wrong-password → 200 no-cookie) is covered by the Go test `TestManagementCookieLogin_ConfigurableAdminPassword` rather than a live probe (a live probe with a wrong password would either lock the account or be blocked as brute-force). B4 (valid-cookie → 200 JSON) is covered by the Go tests that consume the fake.
- The fake reproduces the discriminating BEHAVIOR (cookie gate, status codes, `Set-Cookie` name); it does NOT reproduce Jackett's full API surface — endpoints beyond `/UI/Dashboard`, `/UI/Login`, `/api/v2.0/indexers`, `/api/v2.0/indexers/*/config` return 404. Extending the fake is a follow-up when a new consumer is added.

Cross-references

- Sibling: `docs/scripts/jackett-cookie-login-hardening.md`.
- Fake source: `qBitTorrent-go/internal/jackett/cookie_login_test.go`.
- Client-under-test: `qBitTorrent-go/internal/jackett/client.go`.
- Port doc: `docs/PORTING-FROM-LAVA.md` (P4).
- Constitution: §11.4.107(10) / §11.4.115 / §11.4.201 / §11.4.238.